

Aula 05: Verilog/VHDL Refresher

Agenda

- **PARTE1: HANDS-ON - Examples Verilog + TB (SVlog)**
 - Review Makefile
 - Review FSM (Mealy and Moore Machines)
- **PARTE2: HANDS-ON - FSM**
 - Exercise: Seq Detector in Verilog
 - Exercise: ROBOT
- **VHDL (skipped)**



PARTE I

Exemplos Verilog

> cmd
> docs
v rtl
 1x8demux.v
 8x1mux.v
 adder32.v
 arbiter.v
 counter32bit.v
 decoder2x4.v
 encoder4x2.v
 matrixmult.v
 multiplier16x16.v
 updown_counter.v

v tb
 tb_1x8demux.v
 tb_8x1mux.v
 tb_adder32.v
 tb_arbiter.v
 tb_counter32bit.v
 tb_decoder2x4.v
 tb_encoder4x2.v
 tb_matrixmult.v
 tb_multiplier16x16.v
 tb_updown_counter.v
 Makefile

cmd chore: all of 0x02
docs chore: all of 0x02
rtl chore: all of 0x02
tb adds todo in 0x02
Makefile adds todo in 0x02
TODO.md adds todo in 0x02

README.md

Verilog Circuits Collection – README

This repository contains a set of **basic digital design modules and their testbenches**, written in Verilog. The design sequential logic blocks commonly used in digital systems.

◆ 1. Demultiplexer (1x8) – demux1x8

What it does: Routes a single input (`din`) to one of 8 outputs (`dout`) based on a 3-bit select signal.

Special notes:

- Only one output bit is high at a time.
- Output is **clocked (latched)** even though demux is normally combinational.
- Uses **enable** (`en`) and **active-low reset** (`rstn`).

◆ 2. Multiplexer (8x1) – mux8x1

```
verilog-refresher tree -a
├── .gitignore
├── Makefile
├── cmd
│   ├── 8x1mux.cmd
│   └── arbiter.cmd
├── docs
│   └── README.md
├── rtl
│   ├── 1x8demux.v
│   ├── 8x1mux.v
│   ├── adder32.v
│   ├── arbiter.v
│   ├── counter32bit.v
│   ├── decoder2x4.v
│   ├── encoder4x2.v
│   ├── matrixmult.v
│   ├── multiplier16x16.v
│   └── updown_counter.v
├── scripts
│   └── config-snps.sh
└── tb
    ├── tb_1x8demux.v
    ├── tb_8x1mux.v
    ├── tb_adder32.v
    ├── tb_arbiter.v
    ├── tb_counter32bit.v
    ├── tb_decoder2x4.v
    ├── tb_encoder4x2.v
    ├── tb_matrixmult.v
    ├── tb_multiplier16x16.v
    └── tb_updown_counter.v
```

Exemplos Verilog / Atividade

- 1) Entender infraestrutura com Makefile
- 2) Entender cada circuito
- 3) Simular cada circuitos
- 4) Adicionar *self-checking* em SystemVerilog
- 5) Realizar um relatório final ORGANIZADO e OBJETIVO com:
 - Descrição do estímulo aplicado;
 - Resultados obtidos;
 - Conclusão.
- 6) Enviar PDF p/ mim com título "LAB_A05 <nome do aluno>"

- Na prox avaliação esses circuitos serão usados

Exemplos Verilog

Task 1. Verilog models of reference designs

There are ten reference design modeled using Verilog and their corresponding test bench models.

Task 2. Simulate the design using VCS

Use two step simulation process: Compile and Simulate using following commands

1. `%vcs -file adder2bit_tb.v ../rtl/adder2bit.v`
2. `%.simv`

Task 3. To simulate and debug using Verdi debug environment

For post processing debug, use -debug_access during compile option in following command

1. `%vcs -verilog -debug_access adder2bit_tb.v ../rtl/adder2bit.v`
2. `%.simv`

Note the fsdb file generation after compilation and simulation.

Invoke Verdi debug environment for design debug and viewing waveform of signals by using command:

`%verdi -nologo -ssf file_name.fsdb &`

Question 2. What is the post processing file generated which is used for debug using verdi debug environment?

.....
.....

hands-on

Makefile

- **Makefile**
 - script inteligente de automação baseado em “targets”
- **Evita digitar comandos longos manualmente**
- **Organiza o fluxo:**
 - compilar → rodar → debug → limpar
 - Muito usado em projetos

Makefile

- Estrutura básica

target:

command

- Exemple real no nosso caso:

vcs:

vcs < flags > < arquivos .v >

Makefile

- Como usar?

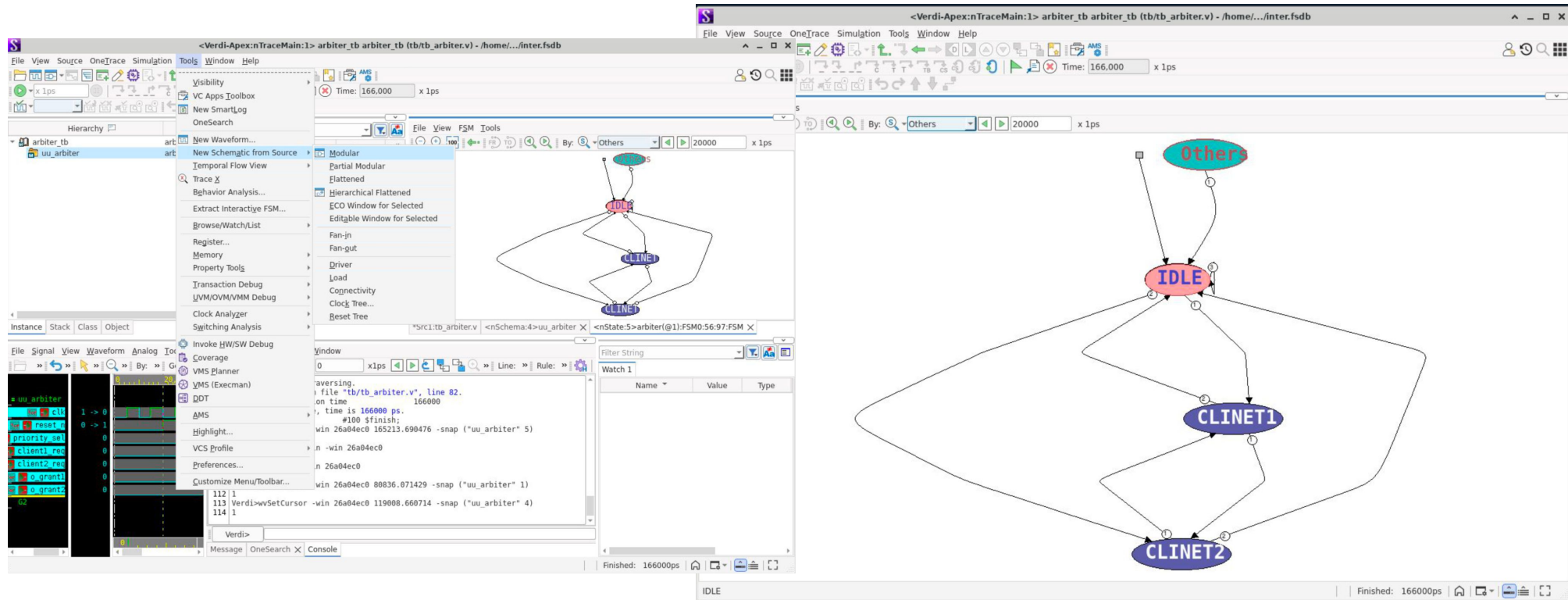
make NOME_DO_TARGET

- Exemplo:

```
Makefile x
prj > 0x02-verilog-ref > Makefile
You, 19 minutes ago | 1 author (You)
1 BLOCK    ?= 1x8demux
2
3 # RTL
4 RTL_FILES = rtl/${BLOCK}.v
5
6 # TB
7 TB_FILES  = tb/tb_${BLOCK}.v
8
9 # VCS
10 VCS_KDB   = -kdb
11 VCS_DEBUG = -debug_access+all
12 VCS_FLAGS = -full64 -l ./vcs_logger.log ${VCS_KDB} ${VCS_DEBUG} -timescale=1ns/1ps
13
14 vcs:
15     vcs ${VCS_FLAGS} ${RTL_FILES} ${TB_FILES}
16
17 run:
18     ./simv
19
20 gui:      You, 25 minutes ago • some verilog codes - week5
21     ./simv -gui
22
23 verdi_sc:
24     verdi -play schematic-top.cmd
25
26 verdi_wf:
27     verdi -play cmd/${BLOCK}.cmd
28
29 vcs_clean:
30     rm -rf csrc *.daidir simv *.log *.key *.conf *.rc *.fsdb verdiLog/
31
```

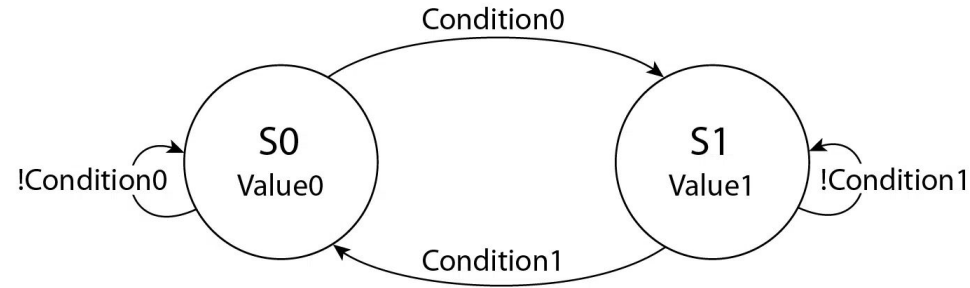
Exemplos Verilog

- É possível observar a FSM no Verdi (eg: Arbiter.v)



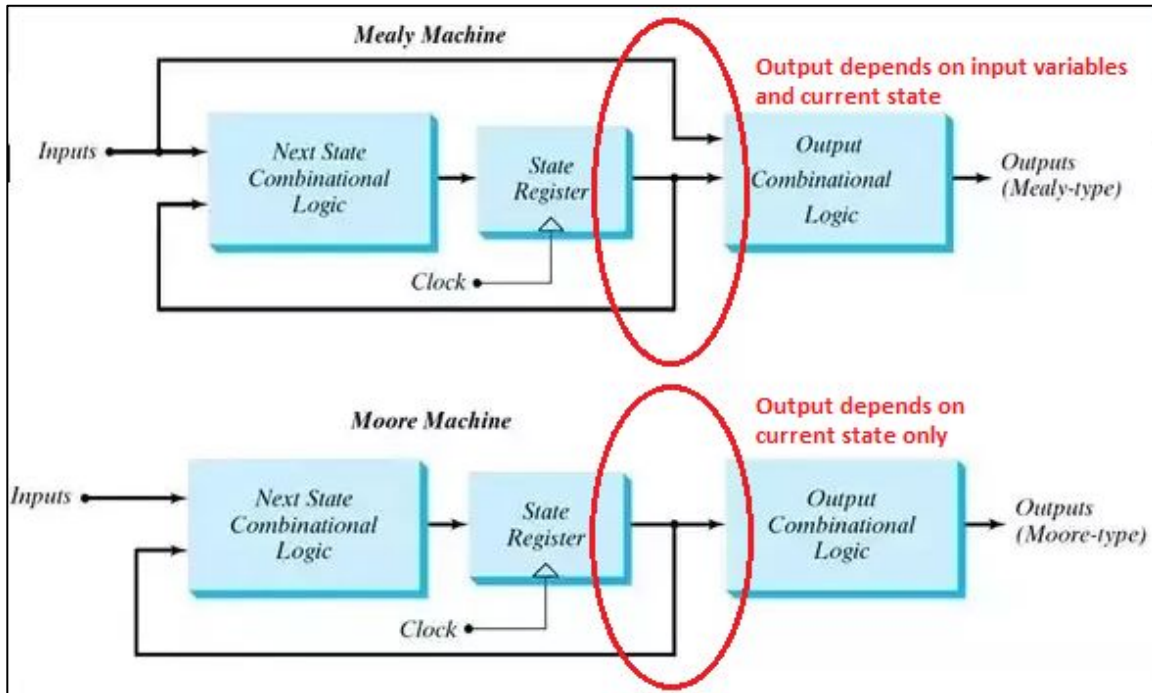
FSM em Verilog

- FSM



FSM em Verilog

- Mealy and Moore





PARTE II



FSM em Verilog / ATIVIDADE

- É uma "receita"



FSM em Verilog

- É uma "receita"



FSM em SystemVerilog

- É uma "receita"
- **Exercício: detector de sequencia 5'b10011**
 - RTL + Testbench

LAB TIME : HANDS-ON!

ROBOT

- **Exercício ROBOT:**
 - Entender Funcionamento
 - Simular as "propriedades do circuitos"
 - Realizar um relatório final ORGANIZADO e OBJETIVO com:
 - Descrição dos estímulos aplicados;
 - Resultados obtidos;
 - Conclusão de cada teste.
 - Enviar PDF p/ mim com título "LAB_A05-P2 <nome>"

ROBOT - REquisitos

Design Requirement

- Objective: To design a robot model which loads and unloads an object to the stations depending on the request
- The sensors connected to the robot sense the request and initiate the correct sequence of operation
- The loading station exists vertically up and keeps loading the Robot after each execution of unloading
- The two unloading stations are located at either side of the Robot
- Assumptions:
 - Robot will service only one sensor request at any point of time



ROBOT - SPEC

Design Specification

- Requirement: The robot under design has 5 degree of movement as in figure
 - Base movement (A)
 - Shoulder movement (B)
 - Arm movement (C)
 - Wrist Movement (D)
 - Claw movement (E)
- It has two possible orientations
- S1, S2, S3 and S4 are four stations and L1 the loading station in the top
- There are corresponding sensors in the Robot to sense the request made by the stations and unloads the data to the respective station
- Each of the joints except E performs 3 actions and require two bit variable to represent the actions
 - Stay
 - Obtuse angle turn (45 ° right)
 - Acute angle turn. (45 ° left)

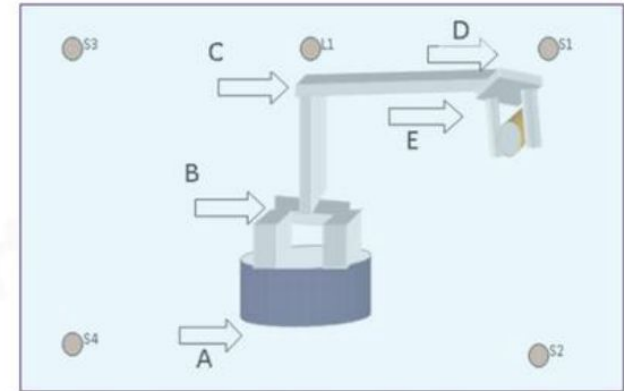


Figure 1 Robot with joints and loading station

A	B	C	Actions
00	00	00	Stay
01	01	01	45 ° right
10	10	10	45 ° left

Figure 2 Actions of wrist during loading

D	Actions
00	Hold
01	Horizontal
10	Vertical

Figure 3 Actions of claw

E	Actions
0	Hold
1	Release

Figure 4 Actions of Joints

ROBOT - Design

Robot Design

- The pin diagram of the model is shown in figure
- Table shows the input output signal descriptions

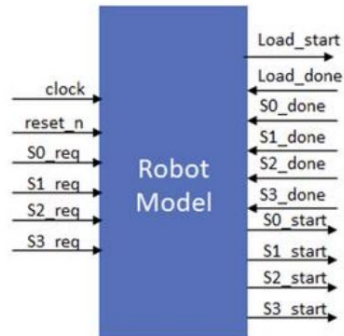


Figure Robot pin diagram

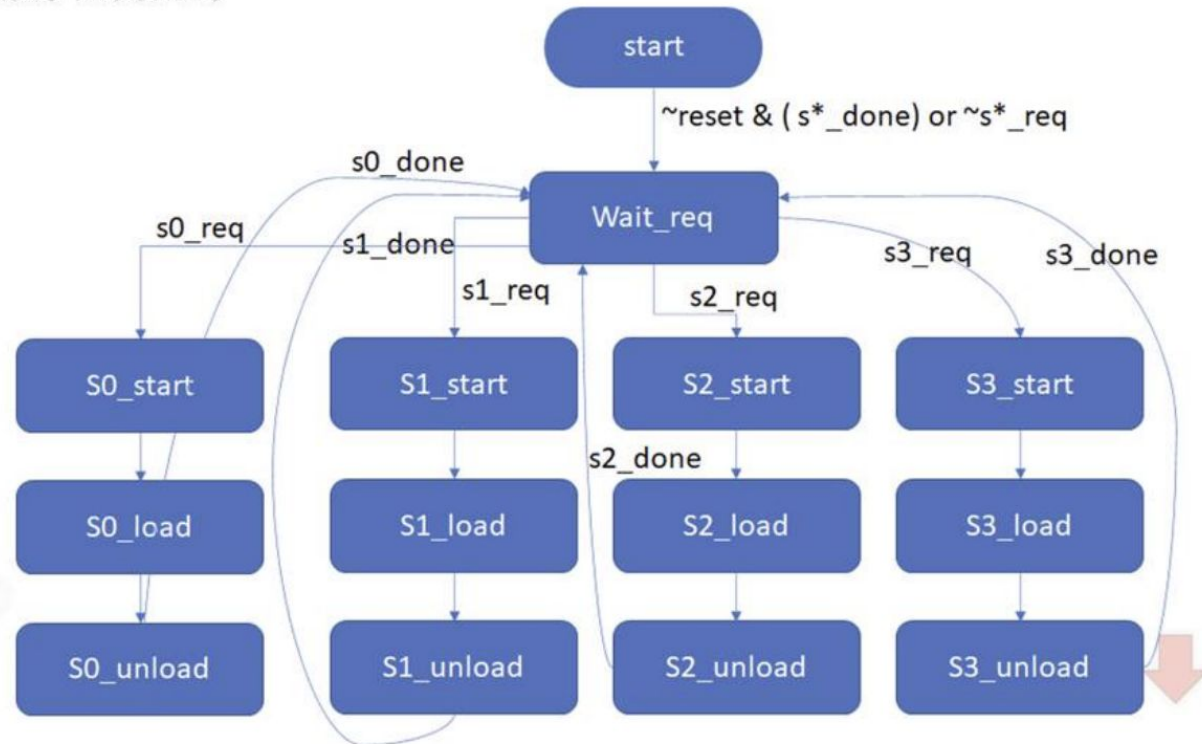
Table : signal description

Signal	IO	Description	Default
clock	Input	Clock input	1'b0
reset_n	input	Asynchronous active low reset signal	1'b1
S0_req	input	Station 1 request for unloading	1'b0
S1_req	input	Station 1 request for unloading	1'b0
S2_req	input	Station 1 request for unloading	1'b0
S3_req	input	Station 1 request for unloading	1'b0
Load_start	Output	Start loading. Whenever requested activity is serviced, the robot will be in initiating loading.	1'b0
Load_done	input	Status indicates the completion of loading.	1'b0
S0_start	Output	Initiate unloading for station 1	1'b0
S1_start	Output	Initiate unloading for station 2	1'b0
S2_start	Output	Initiate unloading for station 3	1'b0
S3_start	Output	Initiate unloading for station 4	1'b0
S0_done	Input	Unloading of station 1 is done	1'b0
S1_done	Input	Unloading of station 2 is done	1'b0
S2_done	Input	Unloading of station 3 is done	1'b0
S3_done	Input	Unloading of station 4 is done	1'b0

ROBOT - Main FSM

ROBOT Main Control State Machine

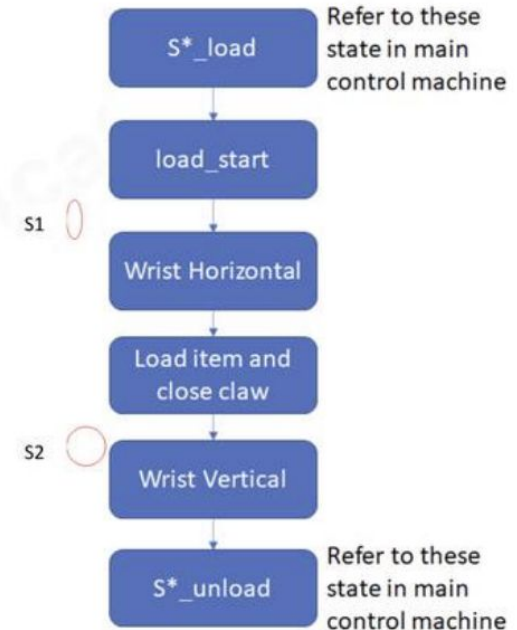
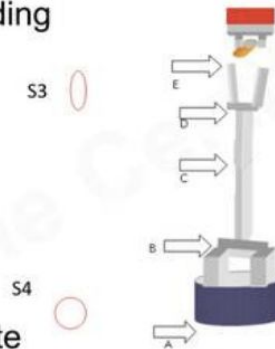
- Robot main control state machine



FSM LOAD

Load State Machine

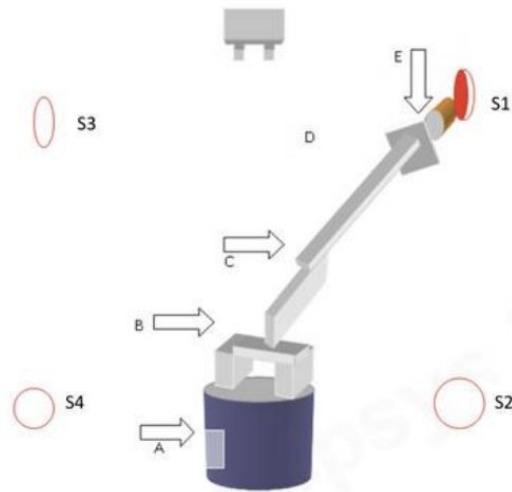
- Load state machine is shown in figure
 - When request from any of stations is received, state machine moves to corresponding s*start state
 - In this state the load_start signal is generated for loading the item to ROBOT
 - Loading is done by three state control which are
 - ◆ Wrist horizontal
 - ◆ Load item and close claw
 - ◆ Wrist vertical
 - Then the control moves to corresponding unload_state where s*_start signal is generated for unloading
 - Unload state machine is triggered from this state



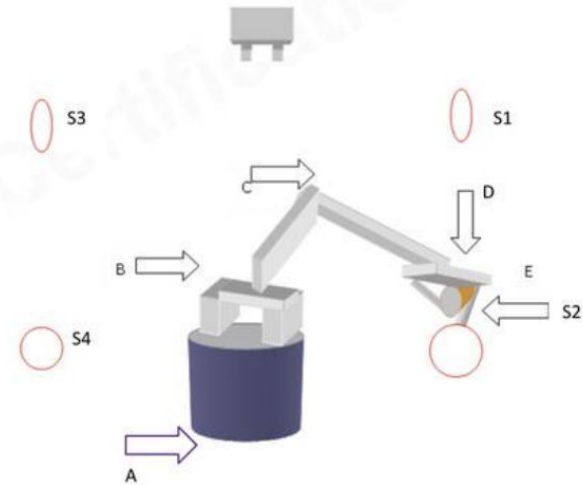
UNLOAD

Unload Operation

Station 1 and 3



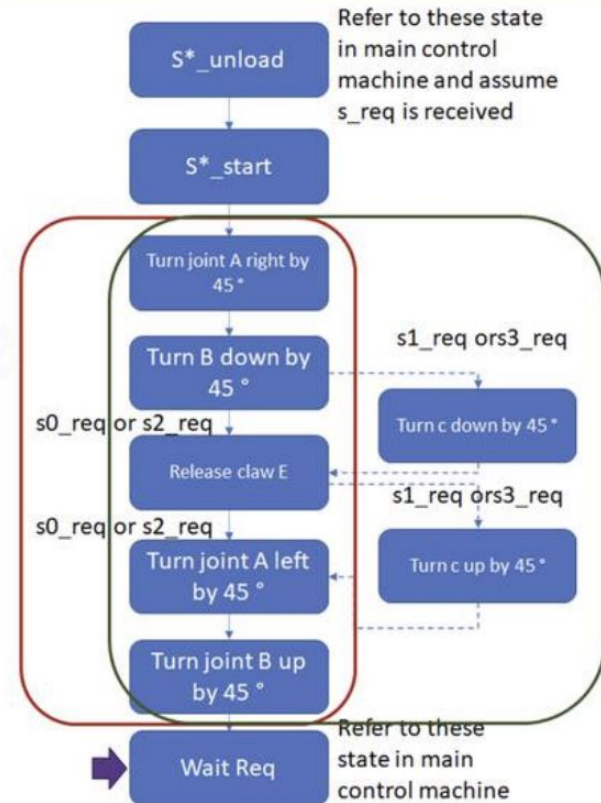
Station 2 and 4



UNLOAD FSM

Unload State Machine

- Unload state machine is shown in figure
 - S^*_start signal is set high for unloading
 - Unloading happens as per the state machine shown
 - For stations 1 and 3, the state machine follows the signals transition as per the straight arrows highlighted by red box
 - For stations 2 and 4, the state transitions follow states in green box. This is due to the additional step of joint C must be turned down and up for these stations as they are down compared to the others
 - Once the unloading is done, state machine goes back to wait_req state in the main control state machine





ROBOT

VERDI SETUP

\$ make robot_comp

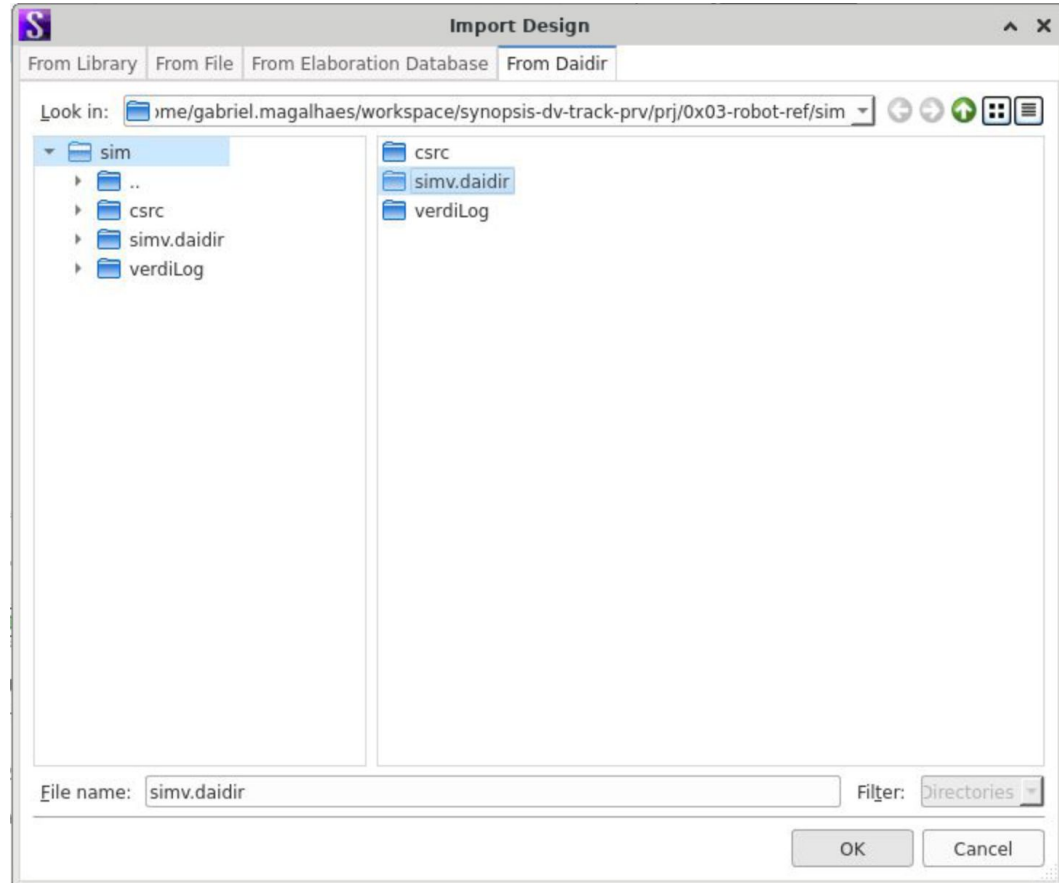
\$ make robot_sim

\$./simv -gui=verdi -l simv.log -sml=verdi

ROBOT

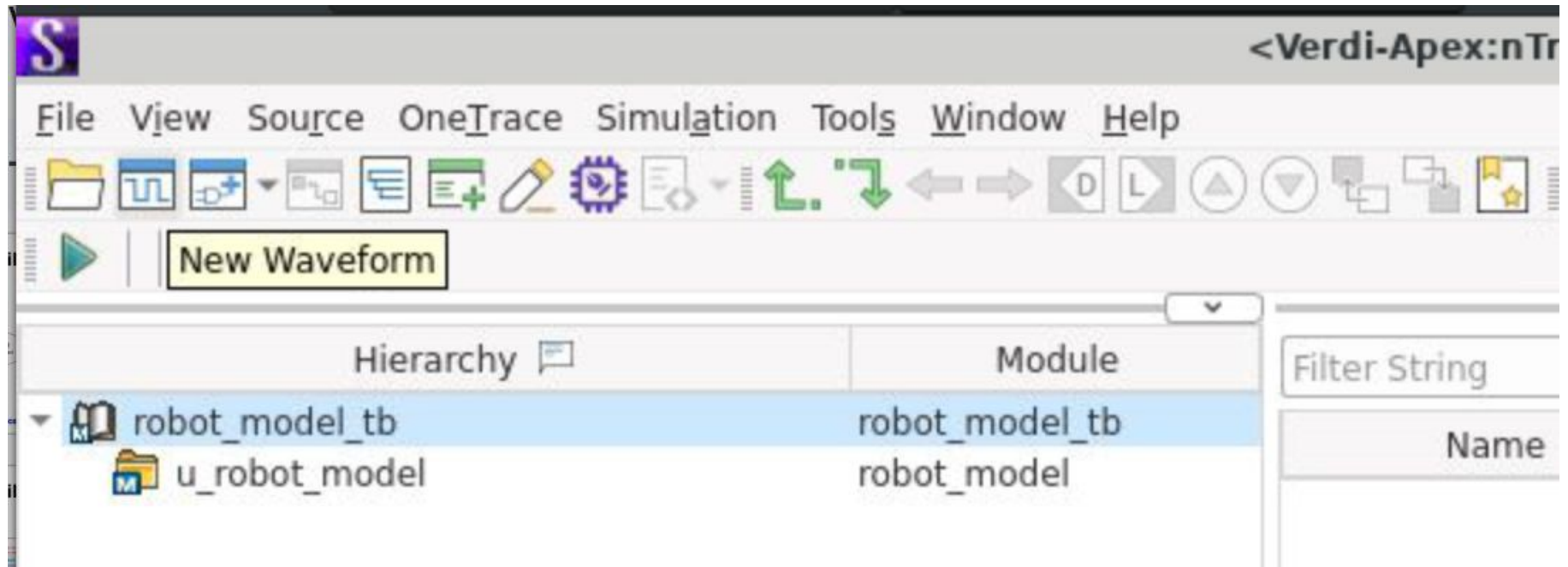
From Daidir

add simv.daidir



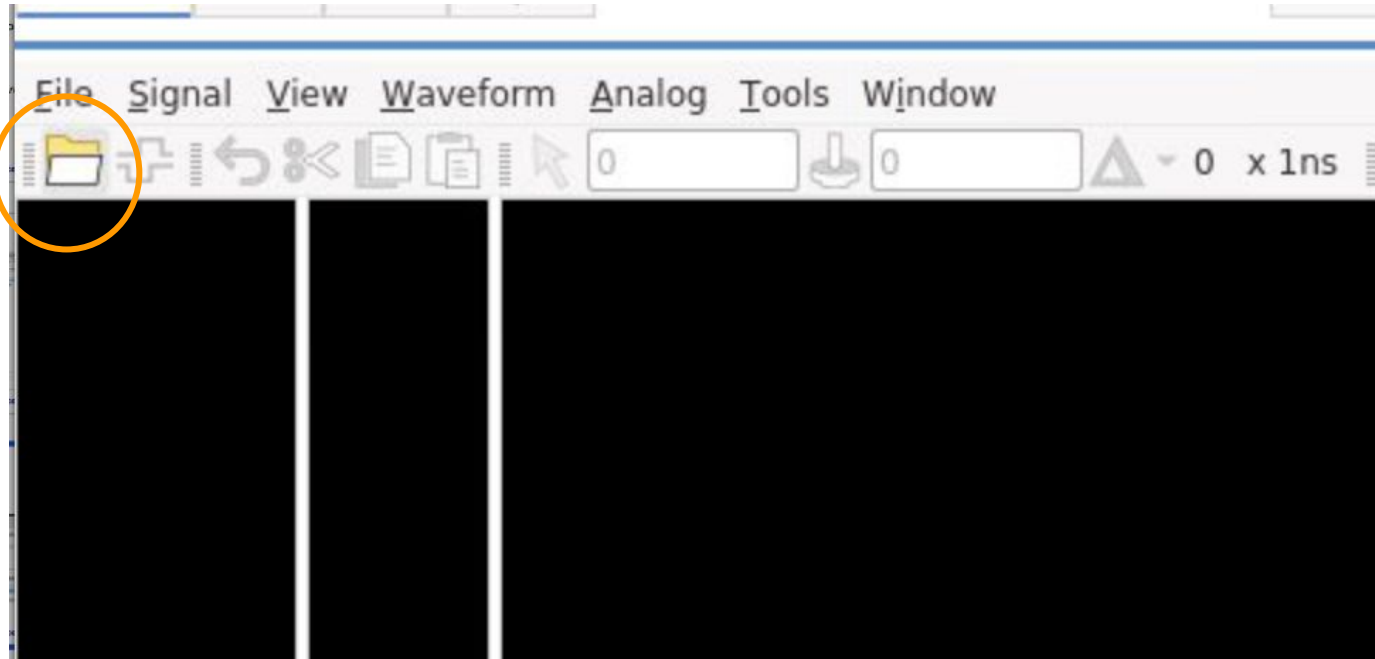
ROBOT

New Waveform



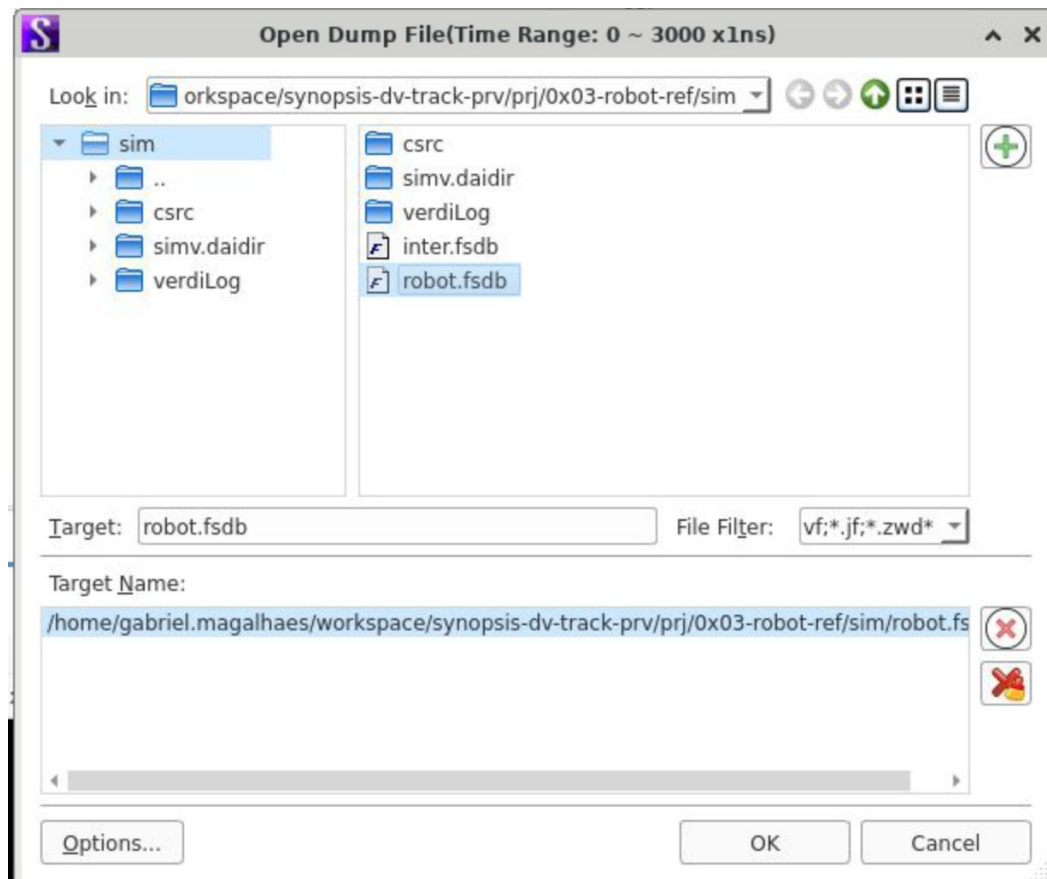
ROBOT

Open Waveform File



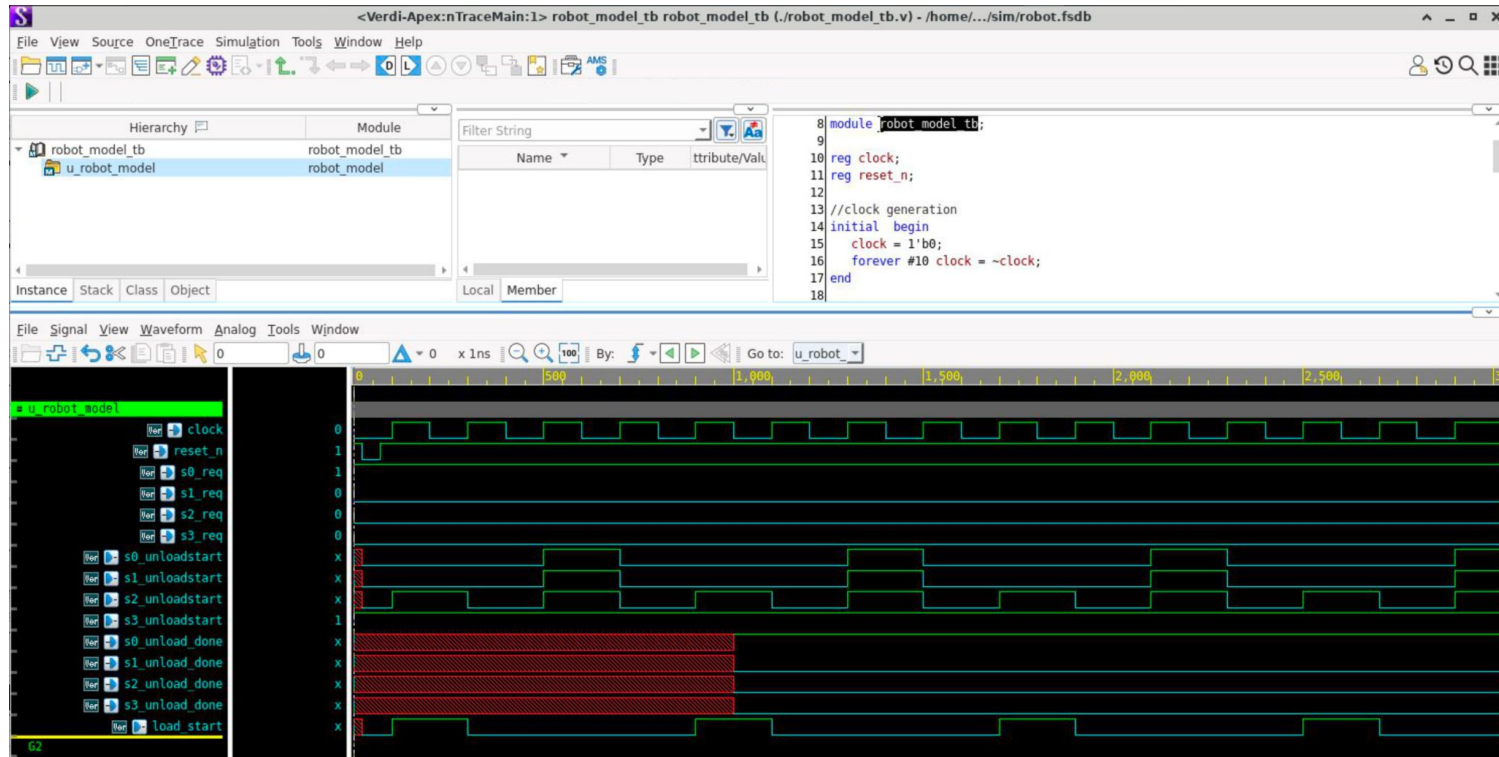
ROBOT

Open
robot.fldb



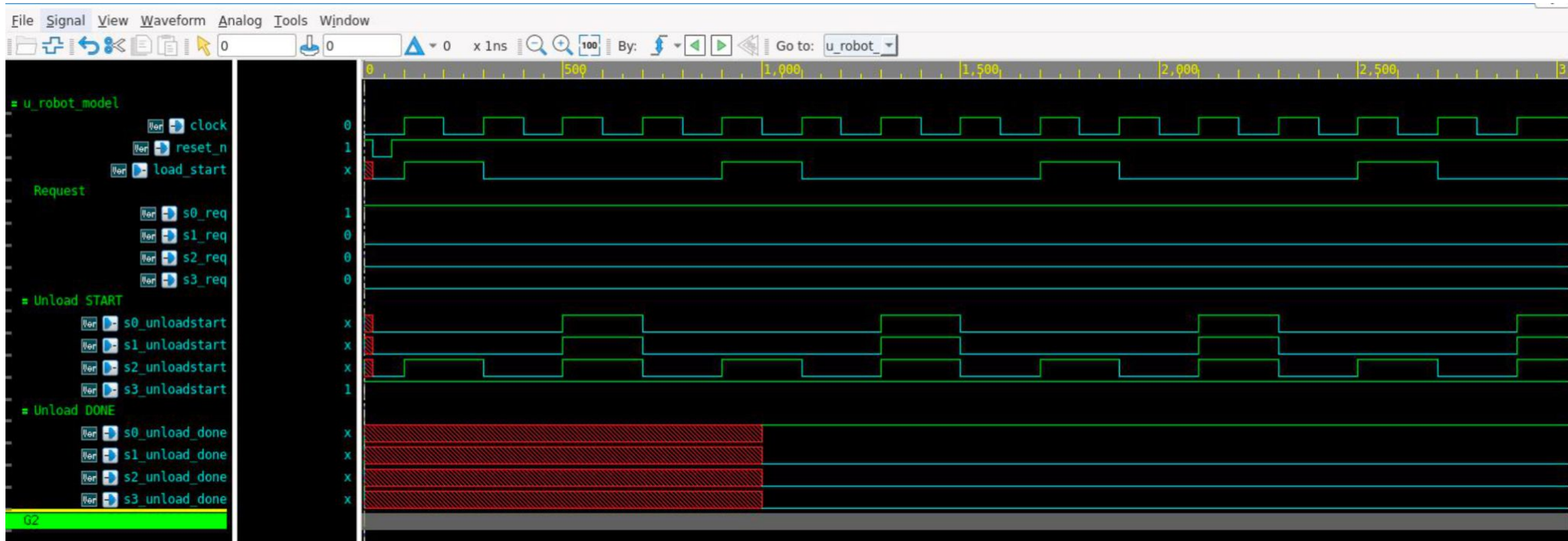
ROBOT

Drag and Drop u_robot_model to Waveform



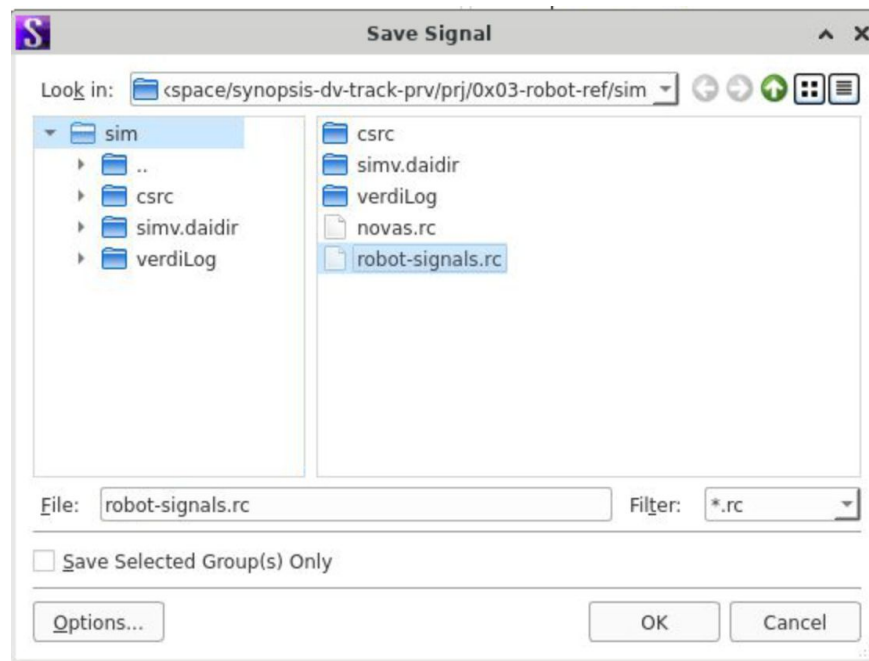
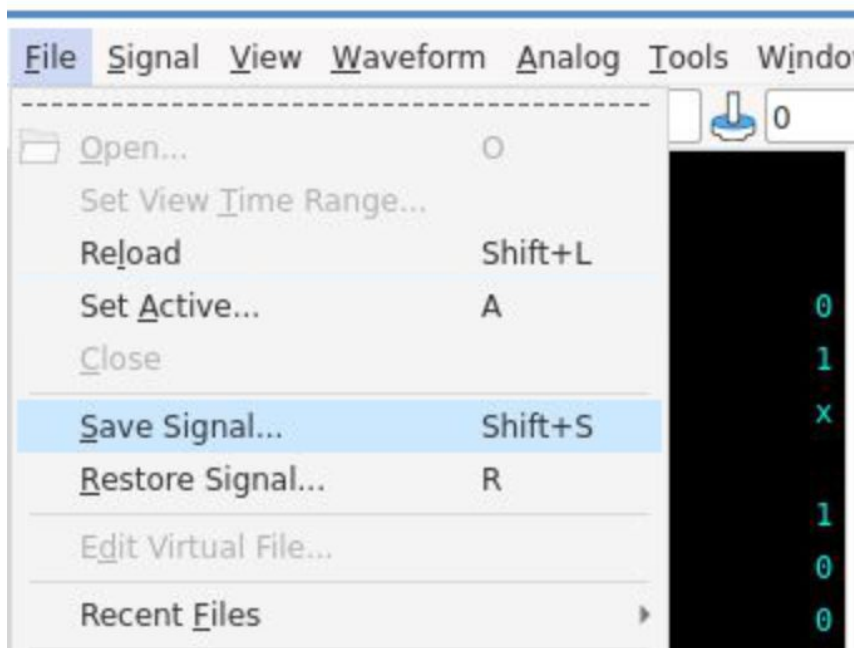
ROBOT

Organize the Signals



ROBOT

Save Signal





ROBOT

Close Verdi and to restart:

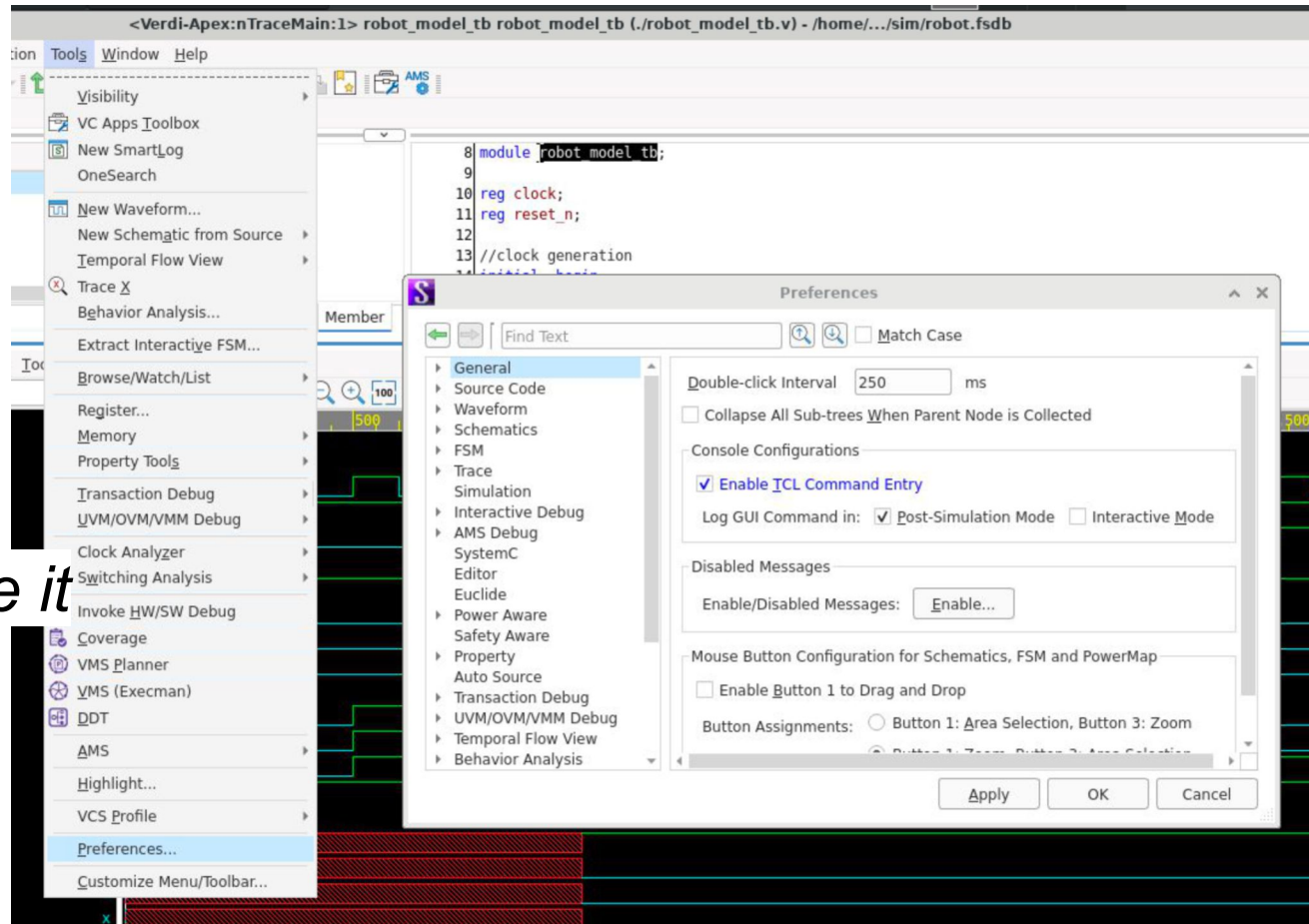
robot_verdi: robot_comp robot_sim

verdi -dbdir ./simv.daidir -ssf robot.fsdb -sswr robot-signals.rc &

ROBOT

*In Tools
Preferences*

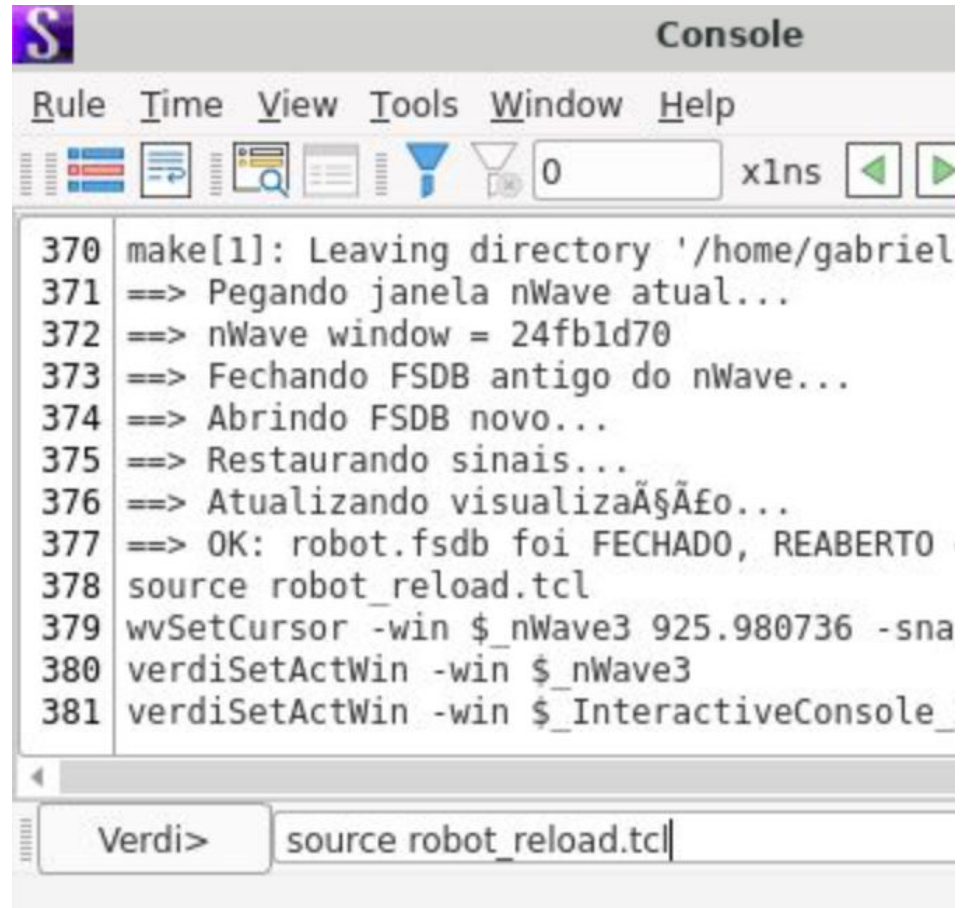
*Enable TCL Cmd Entry
It will open console
if already enabled, ignore it*



ROBOT

To restart simulation:

source robot_reload.tcl



The screenshot shows the VeriWave console window. The title bar is 'Console'. The menu bar includes 'Rule', 'Time', 'View', 'Tools', 'Window', and 'Help'. The toolbar contains icons for file operations, a search icon, and a filter icon. A status bar at the top right shows '0 x1ns' and navigation buttons. The main text area displays the following log output:

```
370 make[1]: Leaving directory '/home/gabriel
371 ==> Pegando janela nWave atual...
372 ==> nWave window = 24fb1d70
373 ==> Fechando FSDB antigo do nWave...
374 ==> Abrindo FSDB novo...
375 ==> Restaurando sinais...
376 ==> Atualizando visualizaÃ§Ã£o...
377 ==> OK: robot.fsdb foi FECHADO, REABERTO
378 source robot_reload.tcl
379 wvSetCursor -win $_nWave3 925.980736 -sna
380 verdiSetActWin -win $_nWave3
381 verdiSetActWin -win $_InteractiveConsole_
```

At the bottom, the 'Verdi>' prompt is visible, followed by the command 'source robot_reload.tcl' entered in the input field.

Applications: <Verdi-Apex:nTraceMai... Terminal - gabriel.maga... X2GO-gabriel.magalhaes-1502-1777239261_stdXfce_dp32

Sun 26 Apr, 18:43 gabriel.magalhaes

<Verdi-Apex:nTraceMain:1> robot_model_tb robot_model_tb (./robot_model_tb.v) - /home/.../sim/inter.fsd

File View Source OneTrace Simulation Tools Window Help

Visibility
VC Apps Toolbox
New SmartLog
OneSearch

Hierarchy

- robot_model_tb
 - u_robot_model

New Waveform...
New Schematic from Source
Temporal Flow View
Trace X
Behavior Analysis...
Extract Interactive FSM...
Browse/Watch/List
Register...
Memory
Property Tools
Transaction Debug
UVM/OVM/VMM Debug
Clock Analyzer
Switching Analysis

Invoke HW/SW Debug
Coverage
VMS Planner
VMS (Execman)
DDT
AMS
Highlight...
VCS Profile
Preferences...
Customize Menu/Toolbar...

File View Schematic Trace Tools Window

Time: 3,000 x 1ns

File View Schematic Trace Tools Window

*S... <nSc... <nState:5>robot_model... <nState:6>robot_model... <nState:7>robot_model...

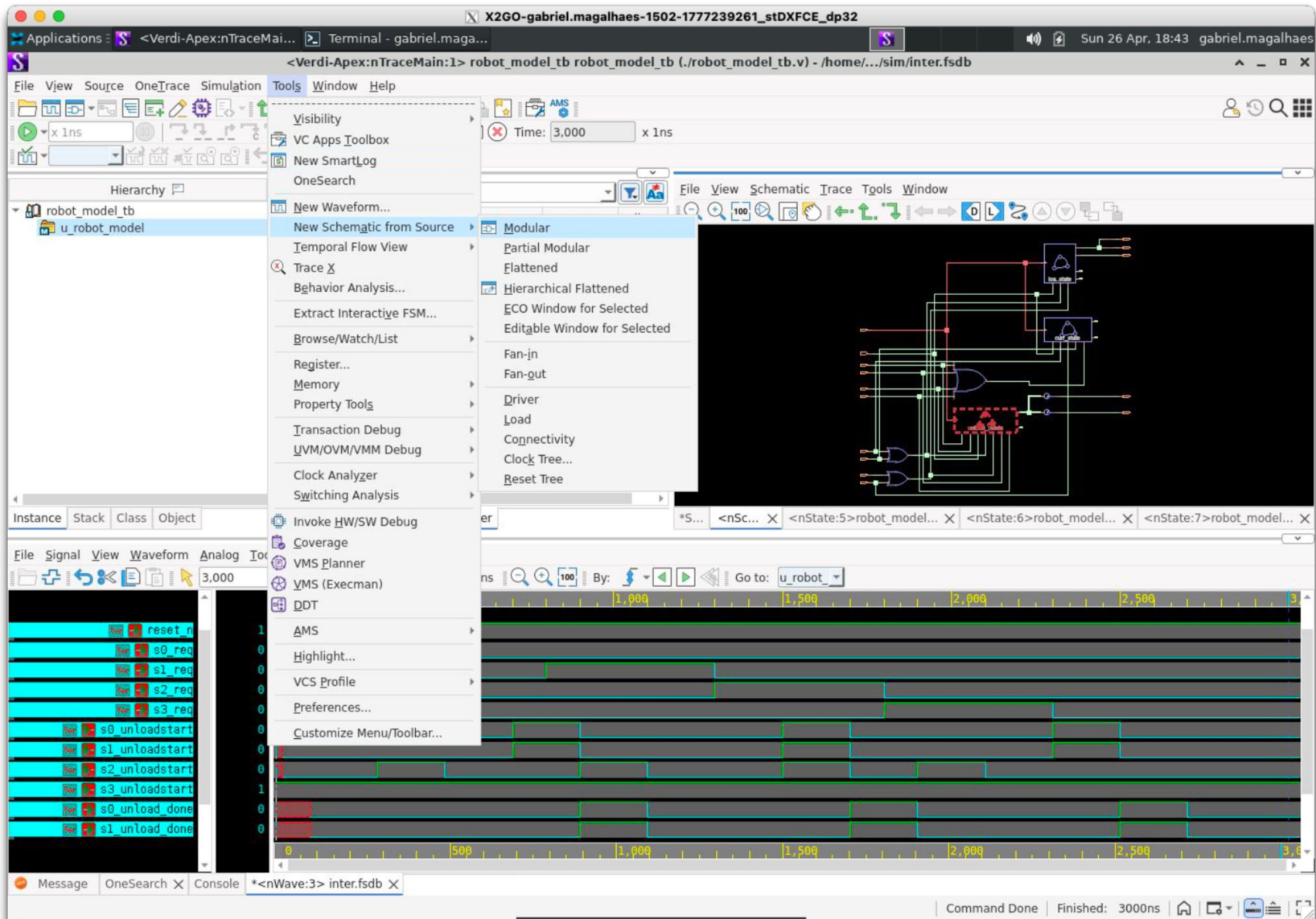
ns 100 By: Go to: u_robot

1,000 1,500 2,000 2,500 3,000

500 1,000 1,500 2,000 2,500 3,000

Message OneSearch Console *<nWave:3> inter.fsd X

Command Done Finished: 3000ns



X2GO-gabriel.magalhaes-1502-1777239261_stDXFCE_dp32

Applications: <Verdi-Apex:nTraceMai... Terminal - gabriel.maga...

<Verdi-Apex:nTraceMain:1> robot_model_tb robot_model_tb (./robot_model_tb.v) - /home/.../sim/inter.fsdb

File View Source OneTrace Simulation Tools Window Help

x 1ns Time: 3,000 x 1ns

Hierarchy Module

- robot_model_tb
 - u_robot_model

robot_model_tb robot_model

Filter String

Name	Type
ttribute/	

File View FSM Tools

By: Others 3000 x 1ns

Instance Stack Class Object

Local Member

*S... <nSc... X <nState:5>robot_model... X <nState:6>robot_model... X <nState:7>robot_model... X

File Signal View Waveform Analog Tools Window

3,000 0 -3,000 x 1ns 100 By: Go to: u_robot_

Message OneSearch X Console *<nWave:3> inter.fsdb X

Selected: (1) Inst: robot_model(@1):FSM2:58:178:FSM

Terminal Emulator Use the command line

Finished: 3000ns

X2GO-gabriel.magalhaes-1502-1777239261_stDXFCE_dp32

Applications: <Verdi-Apex:nTraceMai... Terminal - gabriel.maga...

Sun 26 Apr, 18:40 gabriel.magalhaes

<Verdi-Apex:nTraceMain:1> robot_model_tb robot_model_tb (./robot_model_tb.v) - /home/.../sim/inter.fsd

File View Source OneTrace Simulation Tools Window Help

Time: 3,000 x 1ns

Hierarchy

Module
robot_model_tb
u_robot_model

Filter String

Name	Type
ttribute/	

File View FSM Tools

By: Others 3000 x 1ns

```
graph TD; IDLE((IDLE)) --> 01((01)); IDLE --> 10((10)); IDLE --> Others((Others));
```

Instance Stack Class Object

Local Member

*Src1:robot_model_tb.v <nSchema:4>u_robot_model X <nState:5>robot_model(@1):FSM0:58:101:FSM X

File Signal View Waveform Analog Tools Window

3,000 0 -3,000 x 1ns By: Go to: u_robot_

Message OneSearch X Console *<nWave:3> inter.fsd X

Selected: (1) Inst: robot_model(@1):FSM0:58:101:FSM

Finished: 3000ns

TKS

EXECUÇÃO:



APOIO:



COORDENAÇÃO:

INICIATIVA: